MeetUp 21 September–2026

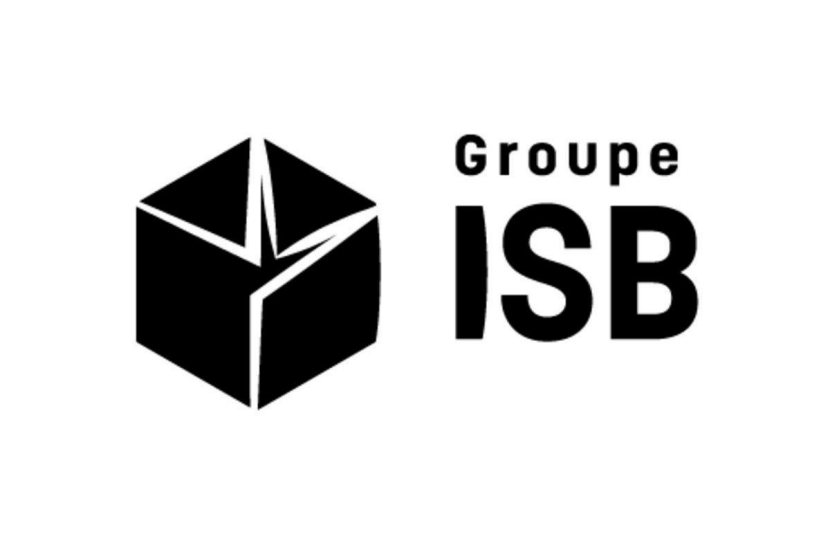
Textual metrics for code quality
&
Software Heritage and CodeCommons

Speaker: Sergiu MOCANU

GitHub: github.com/sergiu-mocanu

Meet the speaker

Currently: Fullstack/AI Engineer Apprentice at ISB Bruz &
Fullstack master's student at MyDigitalSchool





Meet the speaker

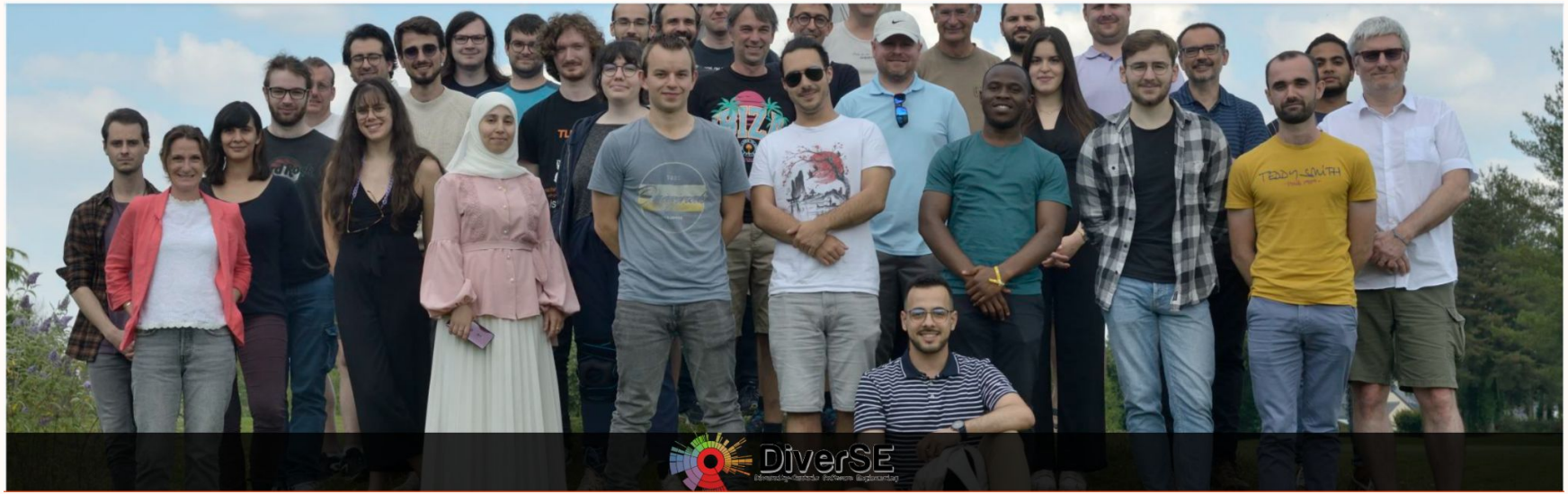
Previously: Bachelor’s & Master’s degree at Rennes 1
2 Internships and 1 CDD contract at Inria/Irisa





Meet the speaker

Internships and research done at DiverSE team



Meet the dream team

Internship supervisors and colleagues



Olivier Barais



Mathieu Acher



Théo Matricon

Second Year of Master's Internship Defense

Sergiu MOCANU

Subject:

The study of *textual-similarity metrics* for **code evaluation**

Research question:

Can textual-similarity metrics replace tests?

Supervisors:

Professors Olivier BARAIS and Mathieu ACHER

Context:

Software security vulnerabilities



Can be found in both





Context: *Black Box Attack*

Input Various Data



Extract and Exploit Information about the Architecture

Observe Changes in the Behaviour

Context: *Code Diversification*



Change the *implementation*.

Preserve the *functionality*.

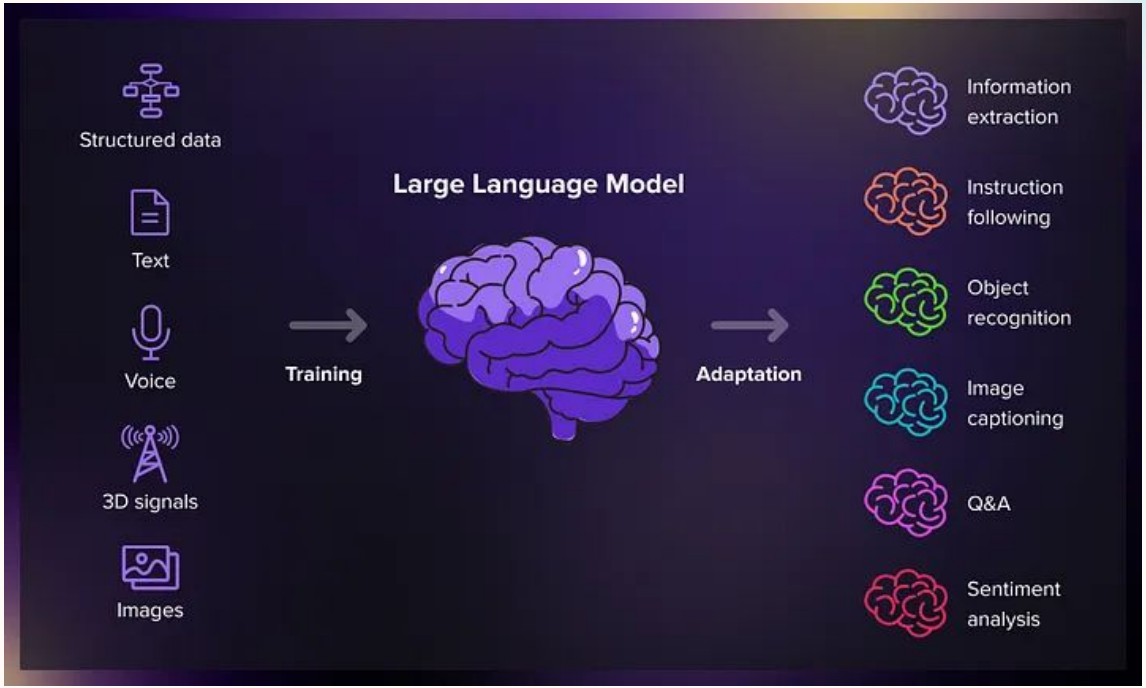
Context:

The problem?

Deterministic tools do not achieve a **desirable** level of code **diversity**.

Context:

One possible solution?



Context: Large Language Models

LLMs are **not** robust—not all the generated code is functional or respects the desired specification.

- Solution: generate multiple implementations and keep only those that successfully *pass* the tests.
- Issue: testing code is very expensive time and energy wise.
- Conclusion: testing all the AI-generated variants is **ineffective** and **unreasonable**.

Motivation: Textual-similarity metrics as a *Test Proxy*

Employing *textual-similarity metrics* as a prefiltering mechanism for the AI-generated code.

Advantages:

- Faster than testing
- Requires less electricity

Background: Textual-similarity metrics

Created to evaluate machine-generated text (e.g., *machine translation*, *automatic summarization*).

They measure the **textual similarity** between one or multiple human-made references and an AI-generated answer.

Background: Textual metric usage example

```
predictions = ["hello there general kenobi", "foo bar foobar"]

references = [
    ["hello there general kenobi", "hello there!"], ["foo bar foobar"]
]
Score: 1.0
```

```
predictions = ["hello there general keno"  "foo bar foobar"]

references = [
    ["hello there general kenobi"  "hello there!"], ["foo bar foobar"]
]
Score: 0.7231269021297695
```

Background: Code-focused similarity metric

```
Prediction:
def add ( a , b ) :
 return a + b

Reference:
def sum ( first , second ) :
 return second + first

Results:
codebleu: 0.5537842627792564
ngram_match_score: 0.10419044640136393
weighted_ngram_match_score: 0.11094660471566163
syntax_match_score: 1.0
dataflow_match_score: 1.0
```

CodeBLEU is a code-specific metric that measures the *weighted n-gram* (e.g., “int”, “var”, “private”), *AST* and *dataflow* similarity.

Motivation: Textual metrics in academic literature

Often, textual-similarity metrics are employed in academic literature in order to compare LLMs' capability in code generation.

Table 2: Metric results for the CoNaLa dataset.

	baseline	tranx-annot	best-tranx	best-tranx-rerank	Codex
BLEU	$12.37^{+1.59}_{-1.46}$	$28.58^{+3.18}_{-3.06}$	$31.48^{+3.01}_{-2.98}$	$33.14^{+2.91}_{-2.94}$	$33.04^{+3.24}_{-3.14}$
ROUGE-L	$36.51^{+1.41}_{-1.46}$	$49.22^{+1.79}_{-1.69}$	$51.47^{+1.87}_{-1.90}$	$52.83^{+1.96}_{-1.84}$	$56.52^{+2.25}_{-2.29}$
ChrF	$17.51^{+1.26}_{-1.26}$	$28.30^{+1.66}_{-1.79}$	$31.14^{+1.89}_{-1.85}$	$32.67^{+2.10}_{-1.95}$	$42.84^{+2.68}_{-2.54}$
METEOR	$28.43^{+1.54}_{-1.54}$	$44.03^{+2.18}_{-2.03}$	$46.55^{+2.28}_{-2.30}$	$48.32^{+2.43}_{-2.38}$	$50.66^{+2.66}_{-2.49}$
RUBY	$43.32^{+1.99}_{-1.84}$	$43.52^{+1.92}_{-1.93}$	$44.81^{+2.00}_{-2.00}$	$46.26^{+2.03}_{-1.97}$	$57.70^{+2.24}_{-2.24}$
CodeBLEU	$30.97^{+1.67}_{-1.47}$	$33.02^{+1.57}_{-1.62}$	$34.07^{+1.67}_{-1.63}$	$34.33^{+1.69}_{-1.65}$	$46.58^{+2.64}_{-2.47}$
Human	$8.74^{+1.80}_{-1.64}$	$26.69^{+3.44}_{-2.91}$	$35.22^{+3.23}_{-3.28}$	$40.10^{+3.70}_{-3.39}$	$59.85^{+3.50}_{-3.50}$

Motivation: Possible textual-metric limitations

Textual metrics might be unsuitable for evaluating code:

- Too **strict** with scripts containing different variable names or order of instructions

- Too **lax** with scripts containing small errors
(e.g., missing parentheses, using an uninitialized variable)

These are just speculations. A rigorous experimental protocol could give us a more definitive answer.

Experimental protocol: Benchmark

For this research we chose the **EvalPlus** benchmark: a set of 164 programming tasks meant to be implemented in Python.

The programming tasks are relatively **easy** to implement, which makes it a good **starting point** for the research.

Experimental protocol: Benchmark

The human-made implementations also contain a **set of tests** meant to verify the correct functionality of the generated scripts.

```
from typing import List

def has_close_elements(numbers: List[float], threshold: float) -> bool:
    """ Check if in given list of numbers, are any two numbers closer to each other than
    given threshold.
    >>> has_close_elements([1.0, 2.0, 3.0], 0.5)
    False
    >>> has_close_elements([1.0, 2.8, 3.0, 4.0, 5.0, 2.0], 0.3)
    True
    """
    assert threshold > 0, "invalid inputs" # $_CONTRACT_$
    assert all([isinstance(v, (int, float)) for v in numbers]), "invalid inputs" # $_CONTRACT_$

    sorted_numbers = sorted(numbers)
    for i in range(len(sorted_numbers) - 1):
        if sorted_numbers[i + 1] - sorted_numbers[i] < threshold:
            return True
    return False


METADATA = {
    'author': 'jt',
    'dataset': 'test'
}

def check(candidate):
    assert candidate([1.0, 2.0, 3.9, 4.0, 5.0, 2.2], 0.3) == True
    assert candidate([1.0, 2.0, 3.9, 4.0, 5.0, 2.2], 0.05) == False
    assert candidate([1.0, 2.0, 5.9, 4.0, 5.0], 0.95) == True
    assert candidate([1.0, 2.0, 5.9, 4.0, 5.0], 0.8) == False
    assert candidate([1.0, 2.0, 3.0, 4.0, 5.0, 2.0], 0.1) == True
    assert candidate([1.1, 2.2, 3.1, 4.1, 5.1], 1.0) == True
    assert candidate([1.1, 2.2, 3.1, 4.1, 5.1], 0.5) == False

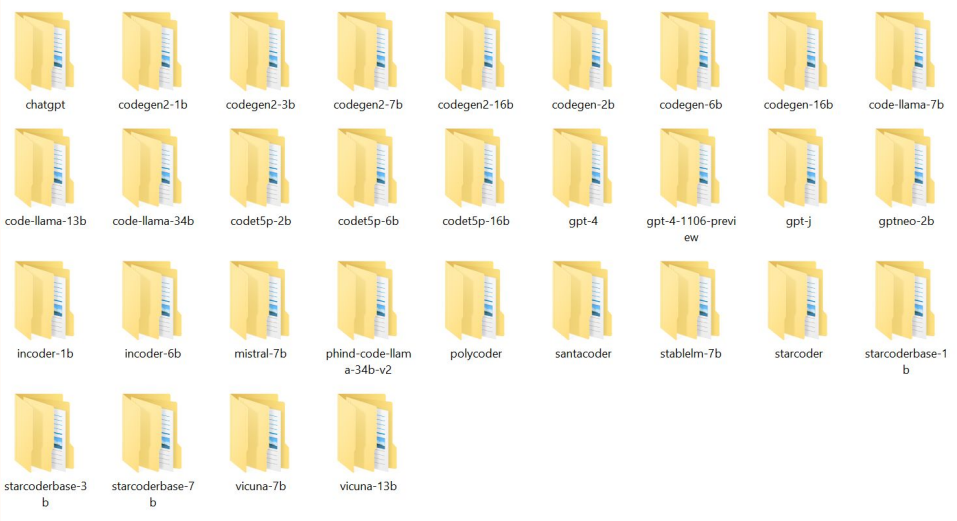
check(has_close_elements)
```

Human-mande implementation of an EvalPlus benchmark task

Set of tests covering all the use cases of the script

Experimental protocol: Benchmark

Additionally, the benchmark offers numerous LLM-generated scripts that implement the benchmark tasks. Numerous AIs were used to generate code–a total of 2,906,453 scripts.



Experimental protocol: Functional testing

The *functionality* test is done through extracting the set of tests from the **canonical** implementation, concatenating it with the **generated** script and executing it in a subprocess.

```
from typing import List
def has_close_elements(numbers: List[float], threshold: float) -> bool:
    for i in range(len(numbers)):
        for j in range(i+1, len(numbers)):
            if abs(numbers[i]-numbers[j]) < threshold:
                return True
    return False

def check(candidate):
    assert candidate([1.0, 2.0, 3.9, 4.0, 5.0, 2.2], 0.3) == True
    assert candidate([1.0, 2.0, 3.9, 4.0, 5.0, 2.2], 0.05) == False
    assert candidate([1.0, 2.0, 5.9, 4.0, 5.0], 0.95) == True
    assert candidate([1.0, 2.0, 5.9, 4.0, 5.0], 0.8) == False
    assert candidate([1.0, 2.0, 3.0, 4.0, 5.0, 2.0], 0.1) == True
    assert candidate([1.1, 2.2, 3.1, 4.1, 5.1], 1.0) == True
    assert candidate([1.1, 2.2, 3.1, 4.1, 5.1], 0.5) == False
check(has_close_elements)
```

AI-generated script

Benchmark test suite

Experimental protocol: Functional testing

The results of the functionality test is saved locally in a JSON file, along the *error type* and *message* in case of a test failure.


```
"5.py": {
  "successful": false,
  "error_type": "NameError",
  "error_message": "name 'log10' is not defined"
},
"6.py": {
  "successful": true
},
"7.py": {
  "successful": false,
  "error_type": "NameError",
  "error_message": "name 'math' is not defined"
```

Experimental protocol: Metric measurement

The next step of the experimental protocol is the measurement of the metric score, which is saved in a CSV file along other relevant information.

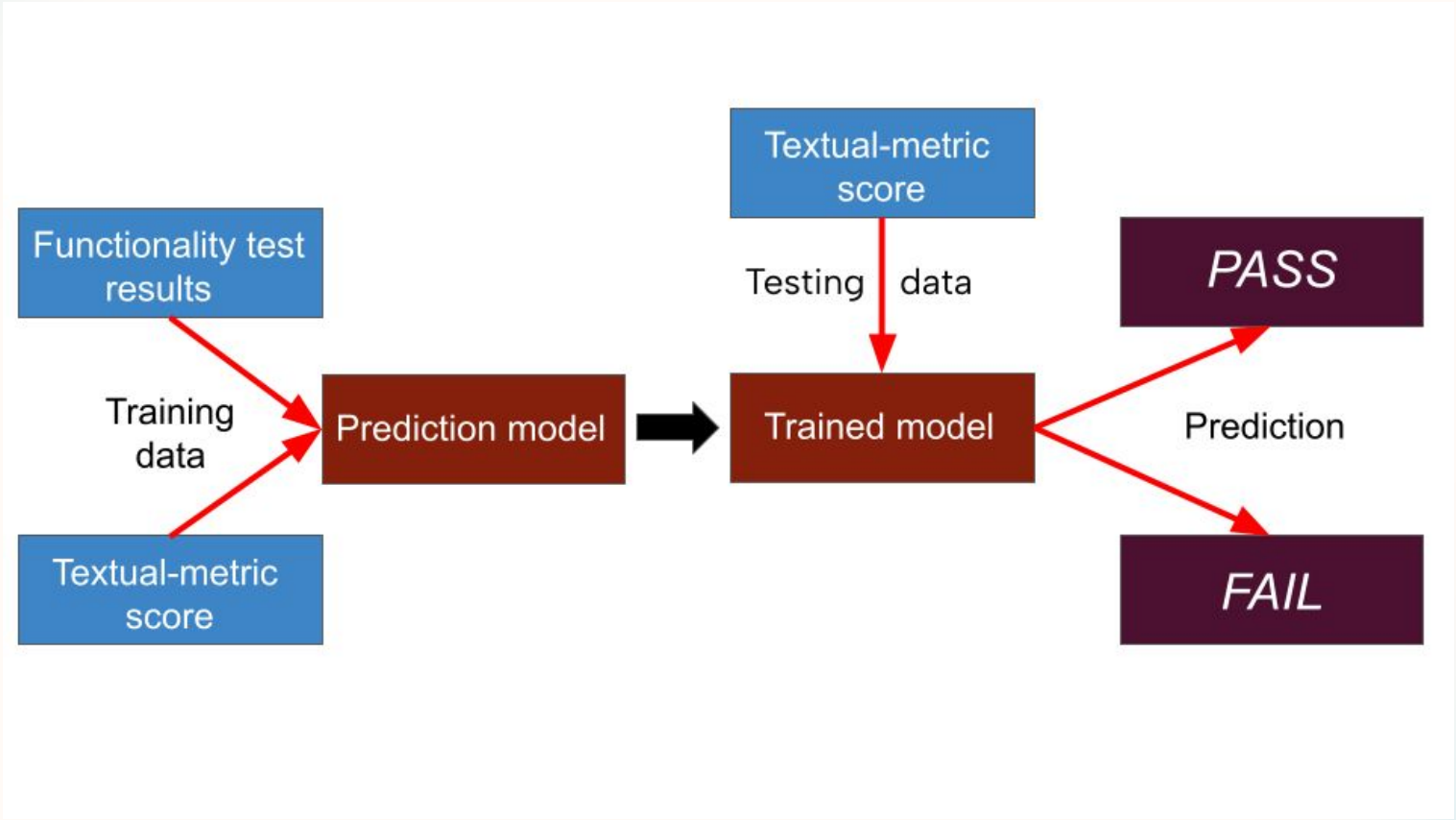
model&temp	script	pass	score
codegen2-1b_temp_0.8	172.py	• True	0.5108111867609832
codegen2-1b_temp_0.8	173.py	• True	0.4389781282875782
codegen2-1b_temp_0.8	174.py	• True	0.4389781282875782
codegen2-1b_temp_0.8	175.py	False	0.43846765462096277
codegen2-1b_temp_0.8	176.py	False	0.4433301411938343
codegen2-1b_temp_0.8	177.py	False	0.4893790667180624
codegen2-1b_temp_0.8	178.py	• True	0.4984618460924052
codegen2-1b_temp_0.8	179.py	False	0.43629072667270546
codegen2-1b_temp_0.8	180.py	• True	0.5548750049988286
codegen2-1b_temp_0.8	181.py	• True	0.4333109868766203
codegen2-1b_temp_0.8	182.py	• True	0.43295583673230537
codegen2-1b_temp_0.8	183.py	• True	0.43263402461482686
codegen2-1b_temp_0.8	184.py	• True	0.4373839058490548

Experimental protocol: Logistic regression

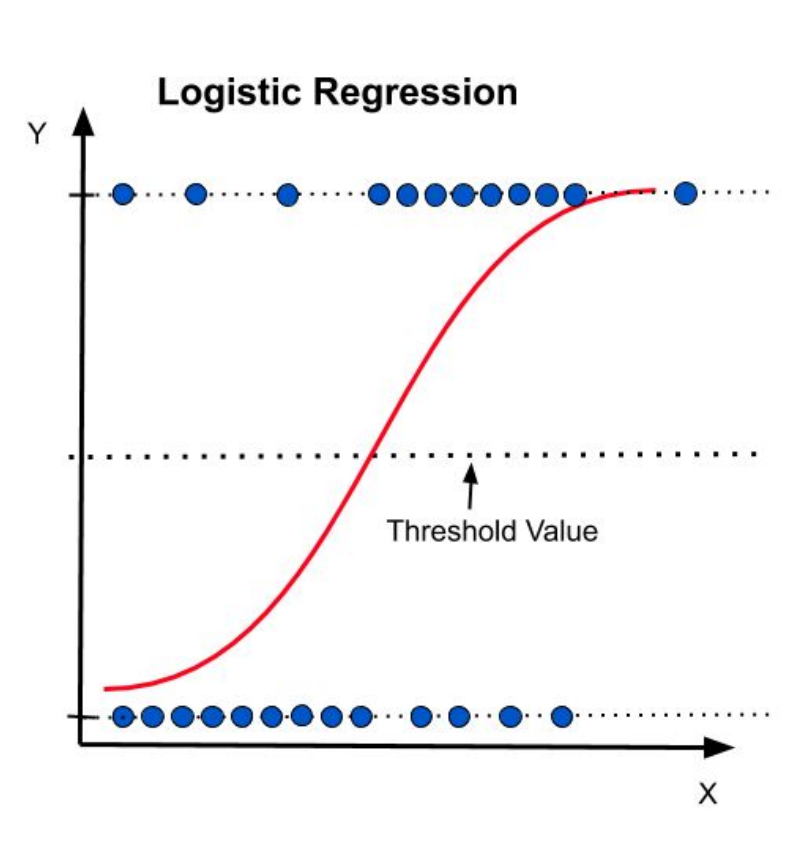
In order to establish a correlation between the metric score and the *pass/fail* label, we employed *logistic regression.*

Logistic regression is a *machine-learning classification model*. It **classifies** objects into two or more classes based on information characterizing the said object.

Experimental protocol: Logistic regression



Experimental protocol: Logistic regression



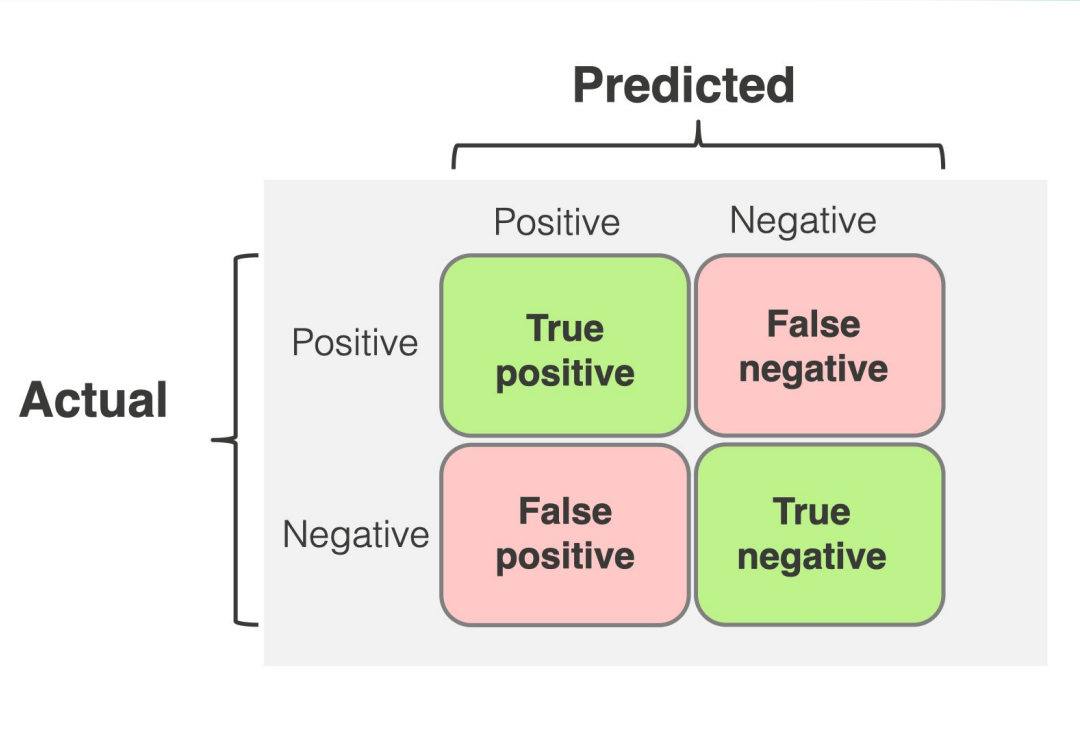
The prediction is done based on a *sigmoid* function and a *probability threshold.* During training, the sigmoid is changed by the model in order to adapt to the training data.

Experimental results: Discussion on benchmark

One flaw of the benchmark is its **unbalanced** distribution of *pass/fail* scripts, with 77% of *fail* scripts.

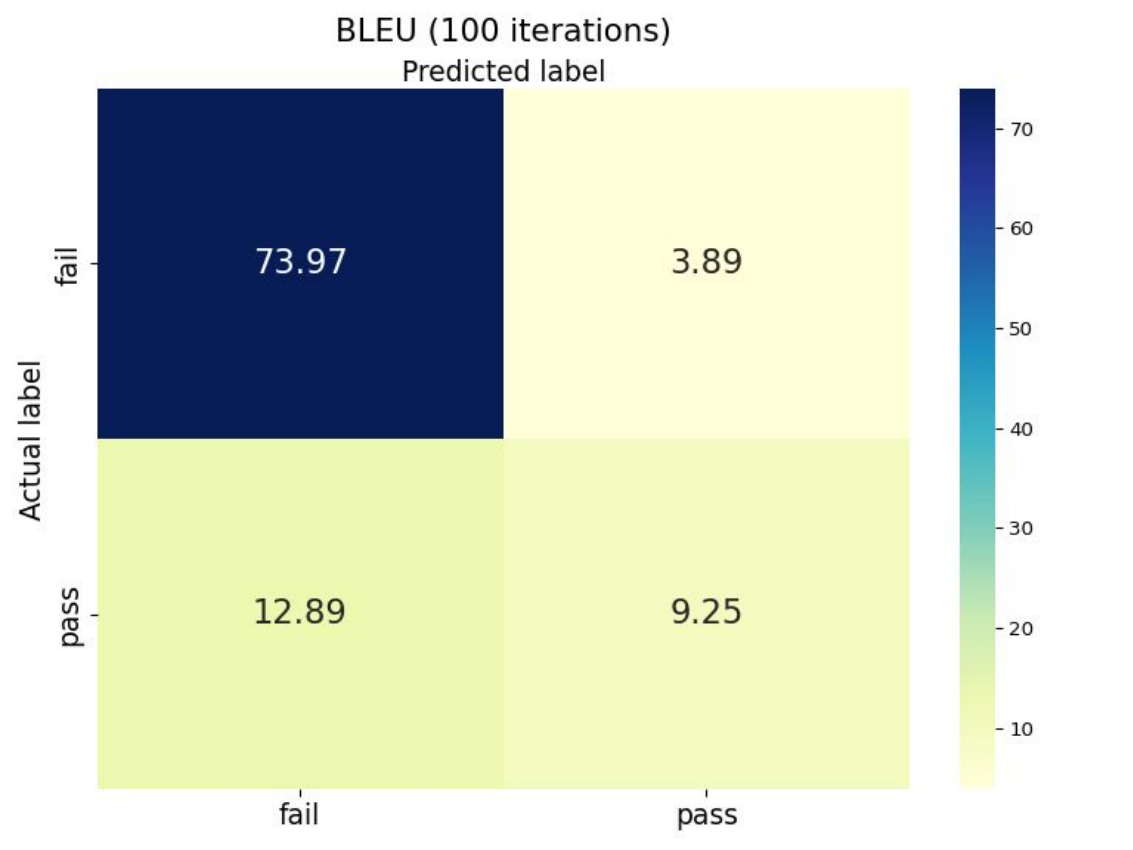
This could lead to training a **pessimistic** prediction model.

Experimental protocol: Confusion matrix

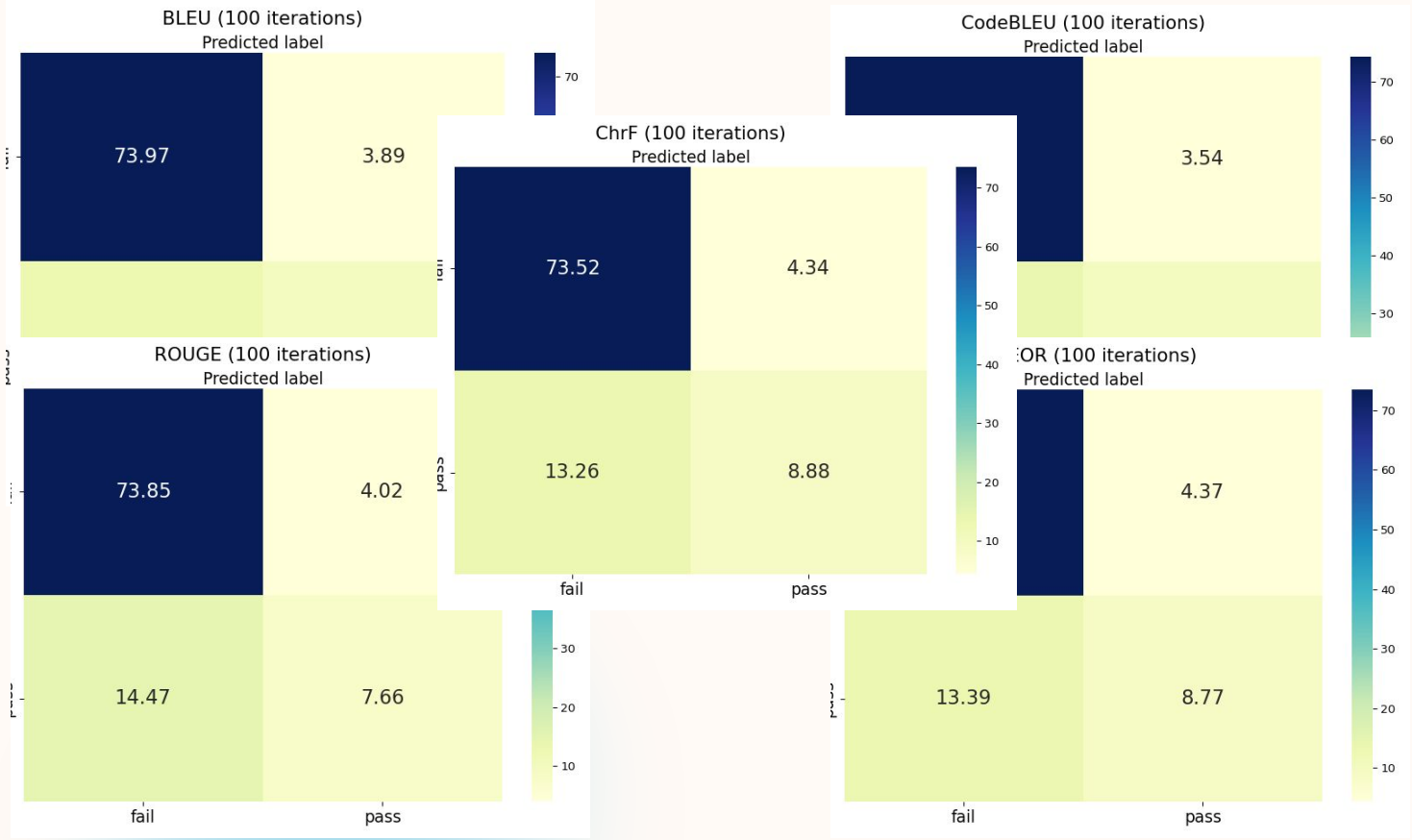


In order to visualize the performance of metric classifiers, we can employ confusion matrices which show the number of correct and false predictions.

Experimental results: BLEU



Experimental results: Performance comparison



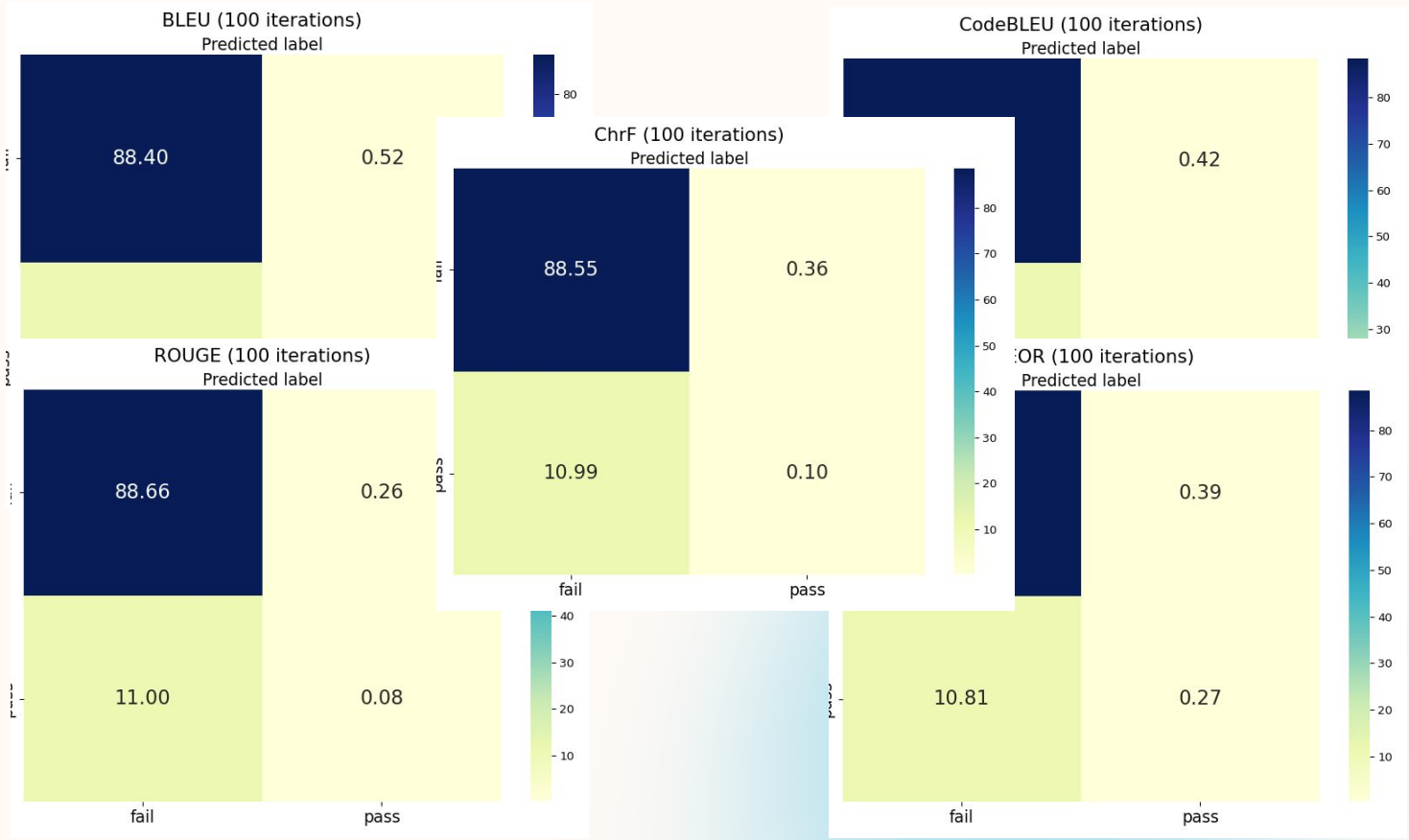
Benchmark observations:

Among the 2,906,453 generated scripts of the benchmark, there are 1,424,695 duplicates.

This leads to model overfitting and results in a biased classifier.

Re-executing the experiment on a cleaned-up dataset might offer different results.

Experimental results: Performance comparison



Experimental results: Discussion on benchmark

One flaw of the benchmark is its **unbalanced** distribution of *pass/fail* scripts, with 77% of *fail* scripts before removing the duplicates, and 87% fail rate after the dataset cleanup.

This could lead to training a **pessimistic** prediction model.

However, the underwhelming performance might be due to the **limitations** of textual metrics.

Experimental results: Textual metrics' limitations

Some coding scenarios include small errors that are **hard to detect** through purely textual analysis:

```python
def separate_paren_groups(paren_string: str) -> List[str]:
    groups = []
    stack = []
    for char in paren_string:
        if char == '(':
            stack.append(char)
        elif char == ')':
            stack.pop()
            if not stack:
                group = ''.join(stack)
                groups.append(group)
        else:
            # Ignore any spaces in the input string.
            continue
    return groups
```

Experimental results: Textual metrics' limitations

```
Prediction:
def add ( a , b ) :
 return a + b

Reference:
def sum ( first , second ) :
 return second + first

Results:
codebleu: 0.5537842627792564
ngram_match_score: 0.10419044640136393
weighted_ngram_match_score: 0.11094660471566163
syntax_match_score: 1.0
dataflow_match_score: 1.0
```

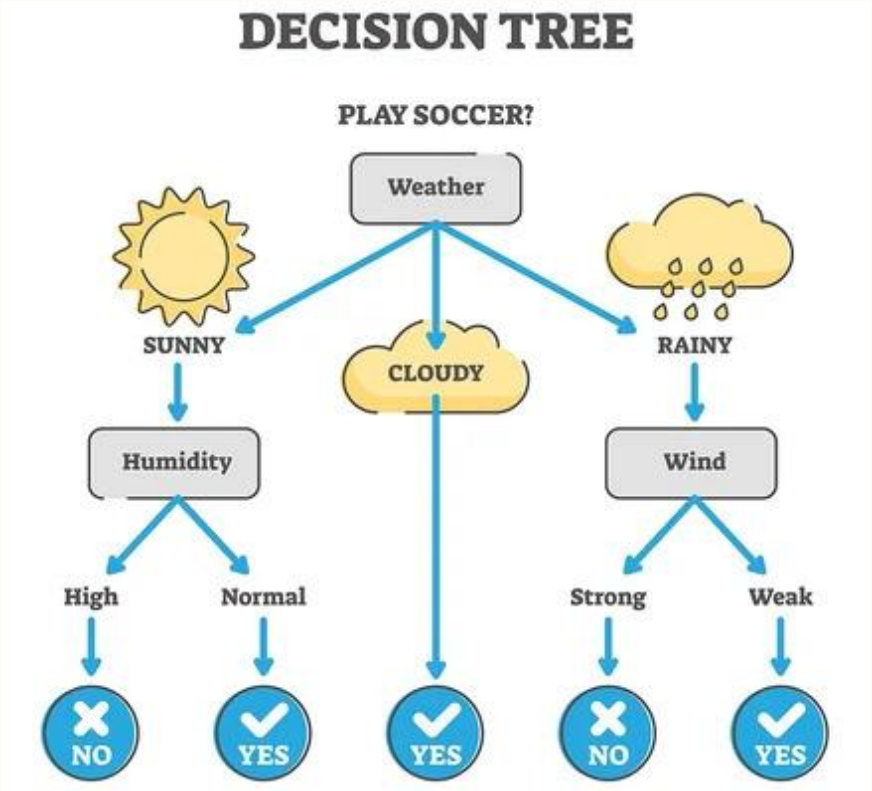
Other scenarios introduce noise that **does not impact** the implementation or the execution of a program.

Experimental results: Textual metrics' limitations

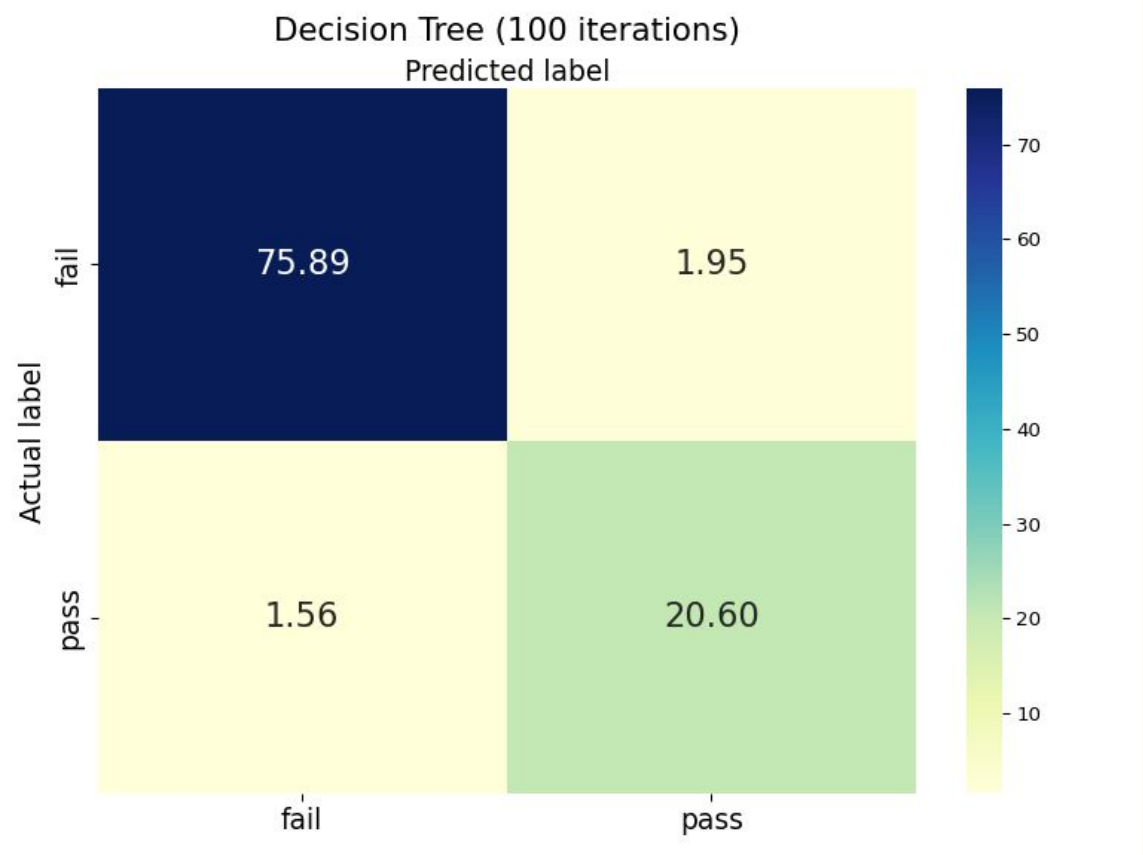
The experimental results show that textual metrics are **unsuitable** for *code evaluation*.

Further research should be done in order to give a more **definitive answer**.

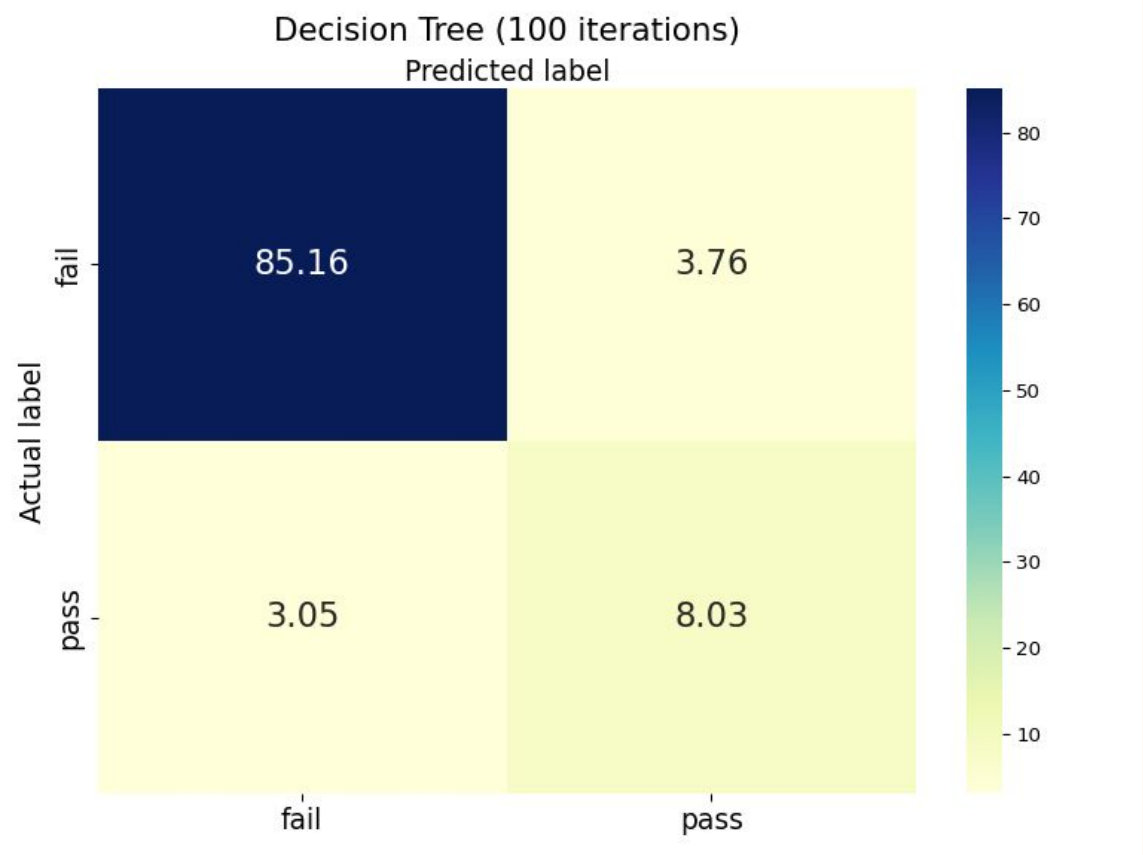
Decision Trees



Decision Trees: All scripts (with duplicates)



Decision Trees: Distinct scripts only



Python Libraries

Scikit-Learn

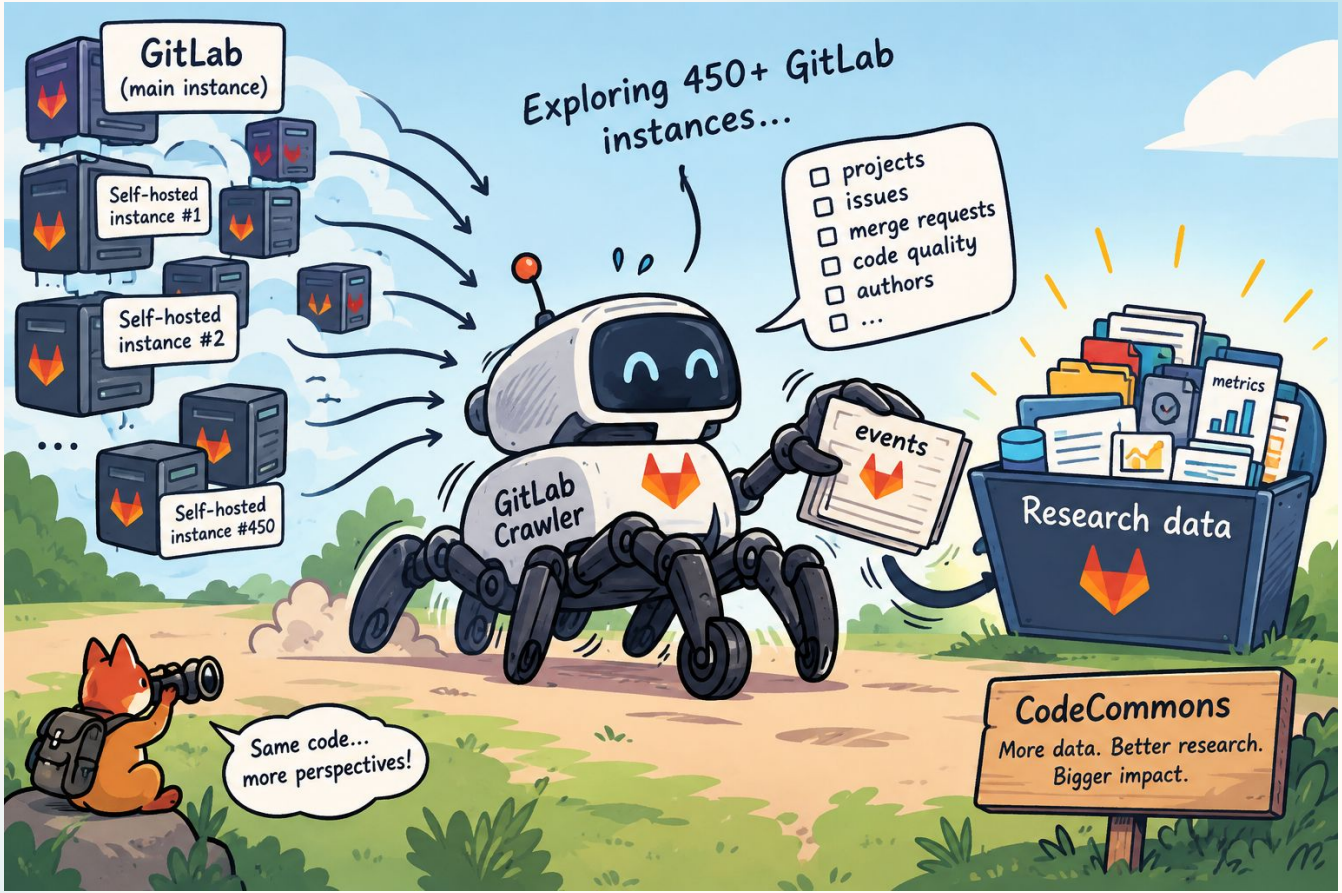
classification, regression and clustering algorithms including support-vector machines, random forests, gradient boosting, *k*-means and DBSCAN

Transformers

Transformers is a Python library that provides ready-to-use implementations and pretrained models based on the Transformer architecture, mainly for AI tasks involving unstructured data such as text, images, audio and video.


The End

Software Heritage
&
CodeCommons

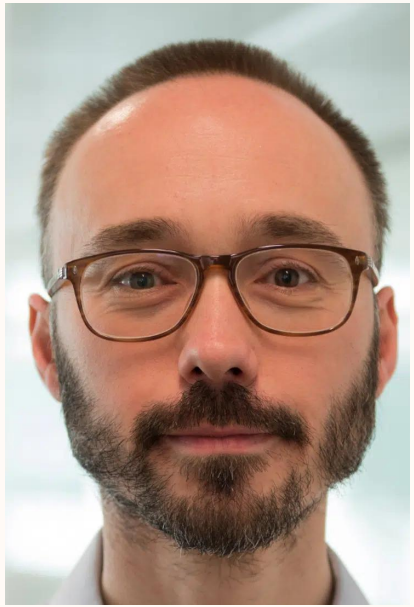

GitLab
(main instance)
Self-hosted instance #1
Self-hosted instance #2
Self-hosted instance #450
Exploring 450+ GitLab instances...
projects
issues
merge requests
code quality
authors
...
GitLab Crawler
events
metrics
Research data
CodeCommons
More data. Better research. Bigger impact.
Same code... more perspectives!

Software Heritage

European initiative created in 2015 by 2 French researchers



Roberto Di Cosmo



Stefano Zacchiroli

Software Heritage

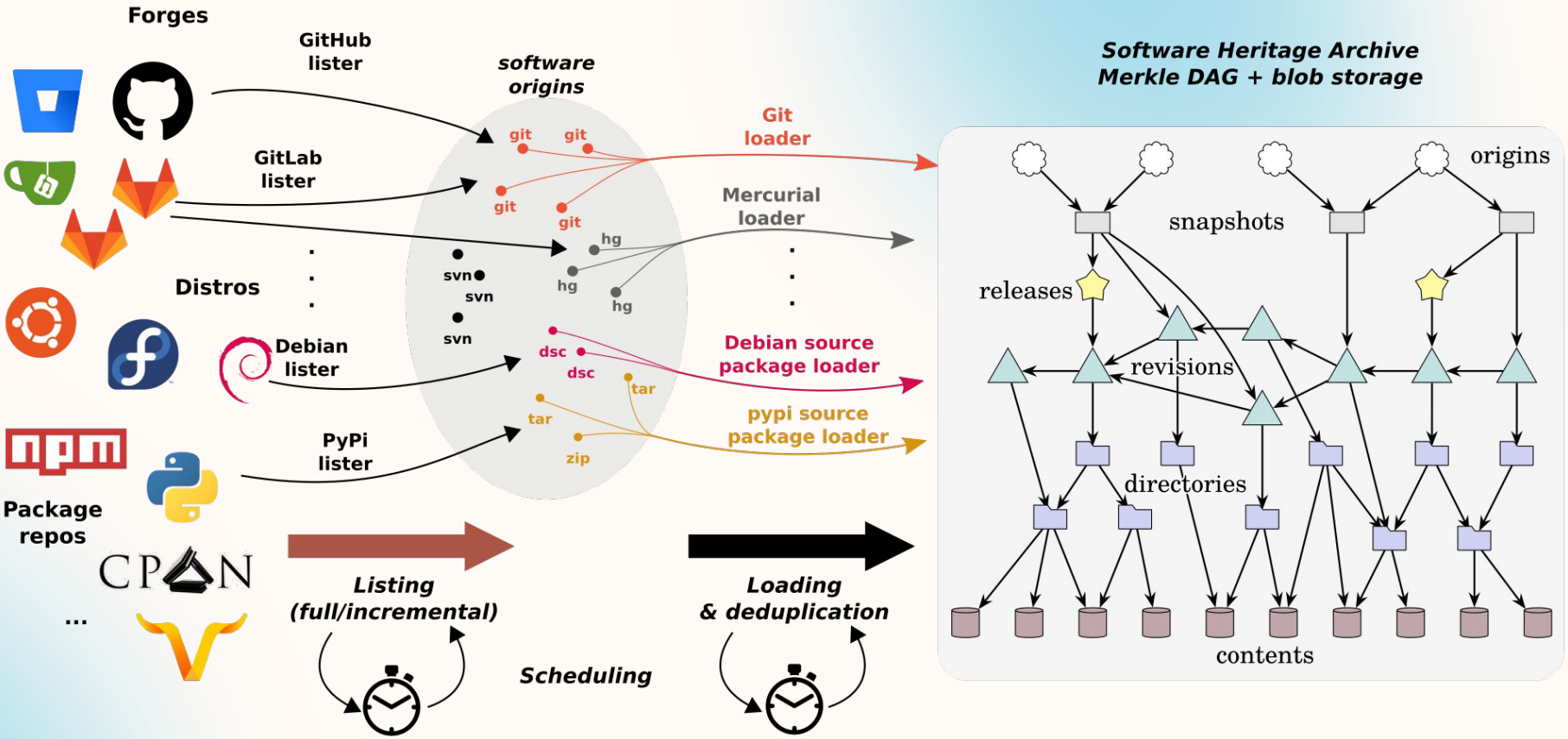


Do we already have your code?

We harvest publicly available source code from many software projects and keep up with development happening there. As of today our archive already contains and keeps safe for you:

Source files	Commits	Projects
29,703,118,947	6,134,881,831	442,296,951

31 Crawled Forges



Logo



Collect



Preserve



Share

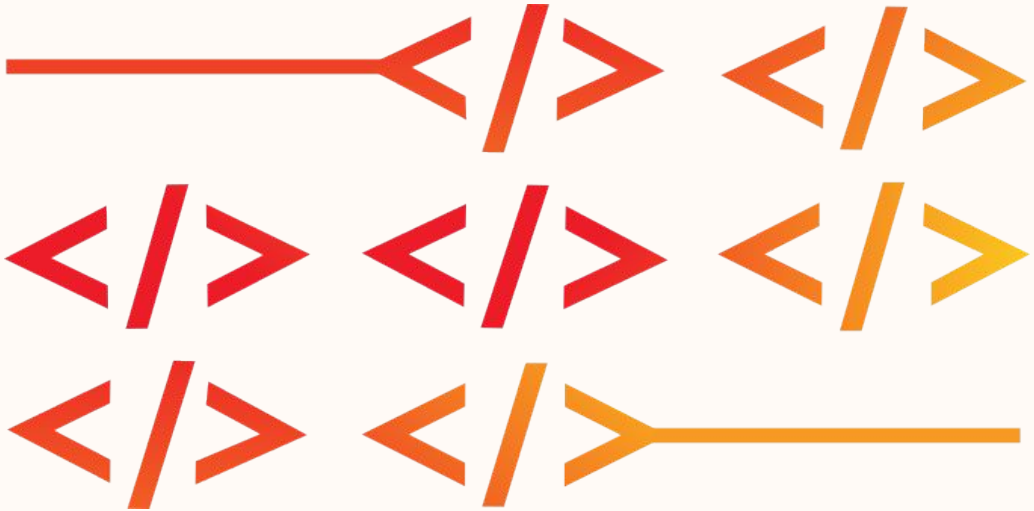
Issue?

Software Heritage (until not long ago) only recovered snapshots of repos

No contextual data – metadata

CodeCommons

Recover all available metadata:
Contributors, Issues, Discussions, Pull Requests, etc.



Better training for code-specialized LLMs?

Truly open-source LLM



Against training AI models?

MIT No Attribution / MIT No-AI-style licenses

Other uses for Software Heritage



Heritage

Software is an important part of human production. It is also a key enabler for salvaging our entire digital heritage.

We collect, preserve, and make accessible source code for the benefits of present and future generations.



Science

Science relies more and more on software. To guarantee scientific reproducibility we need to preserve it.

Amassing source code at this scale will be challenging, but will also enable the next generation of software studies.



Industry

Software is present in all industrial processes and products.

The universal source code archive we are building will help industry with provenance tracking, long-term archival, and software bill of materials.

Other uses for Software Heritage

- **Reproducibility:** researchers can refer to a preserved version of source code rather than relying on a repository remaining online and unchanged.
- **Security:** historical versions make it possible to investigate how vulnerabilities appeared and evolved.
- **Education:** researchers and students can study real-world code at enormous scale.
- **Abandoned/open-source projects:** preservation doesn't depend on a project remaining actively maintained.
- **Dependency preservation:** software often depends on other software that can disappear; preserving the ecosystem helps maintain historical context.